SMART STUDY RECALL / ENGINEERING AND RESEARCH GUIDE

Next-Session Model and Review Policy

HLR, baselines, GRU, evaluation, prediction serving and the 80% scheduler

Prepared from the repository implementation and experiment reports as inspected on 27 October 2026.

Faculty presentation edition | Implementation facts are separated from research hypotheses and unvalidated claims.

How to use this document

This guide explains the running implementation, what data enters each component, what algorithms do with it, what is saved, how to operate the project, and what the current experiments do and do not establish. Formulae use plain ASCII notation. Source paths at the end let a reviewer trace each description back to the repository.

Important interpretation rule: a predicted next-session correct fraction is a score proxy. It is not a measured probability that a learner will remember a course concept on a future date.

Contents

01. 1. Prediction target and boundary

02. 2. Feature construction

03. 3. Candidate models

04. 4. Training, split design and deployment selection

05. 5. Evaluation and actual recorded results

06. 6. Serving path: from event to review date

07. 7. Limits and the right next experiment

1. Prediction target and boundary

The deployed next-session model estimates the fraction of correct answers in a learner-item's next observed session. Given ordered history through the current session and a proposed gap g (days), it returns y_hat = next_session_correct / next_session_seen. The gap is an input scenario. The model itself does not choose the review date.

The app trains and evaluates on Duolingo Learning Traces, a language-learning dataset with user IDs, lexeme IDs, timestamps, history counts and session counts. The exact task is not a delayed quiz about an app course topic. It has no direct labels for when a learner forgets a fact from uploaded material.

Keep the predictor and policy conceptually separate: Predictor = expected next-session score conditional on gap. Policy = select an interval under an explicit threshold. The date is a policy output derived from a score curve, not a direct target label.

2. Feature construction

For each learner-item, rows are ordered by timestamp. Ambiguous tied user-item timestamps are removed in this experiment because the source does not specify their internal ordering. A row is paired with the next row for the same learner and item; the difference between timestamps becomes the observed candidate gap, limited to 30 days.

Feature	Construction	Intuition
log_updated_seen	ln(1 + history_seen + current_session_seen)	Exposure count with a compressed scale
updated_accuracy	(history_correct + current_session_correct) / updated_seen	Cumulative performance through current session
log_session_seen	ln(1 + current_session_seen)	Current session evidence size
session_accuracy	current_session_correct / current_session_seen	Current session outcome
log_candidate_gap_days	ln(1 + proposed gap in days)	Time until the hypothetical next observation

For each scoring request, the adapter rebuilds the ordered feature rows from session counts. The final observed gap is intentionally ignored; it is replaced with the scenario gap being evaluated. A sequence length of 10 is used for the GRU. Short histories are left-padded by repeating their earliest row.

3. Candidate models

HLR: half-life regression

The HLR model maps cumulative exposure and accuracy into a half-life H, then applies exponential forgetting form. In the trained candidate, log(H) = beta0 + beta1 * log_updated_seen + beta2 * updated_accuracy. Predicted score p = 2^(-g/H). The currently selected artifact is kind=hlr with fitted coefficients approximately beta0=0.625, beta1=-0.502 and beta2=4.360. The training run can select different coefficients after data or split changes.

This implementation of HLR does not directly learn a course-specific neural state. It is a small parametric curve: more accurate history raises the estimated half-life; the learned exposure coefficient controls the effect of cumulative seen count. Since p is fit to next-session scores on a proxy dataset, its shape should not be interpreted as a calibrated memory probability.

Mean baseline

Predict a constant mean score learned from the training partition. It is a useful floor: a complicated model should beat this simple average on held-out outcomes.

Linear regression

Fit an ordinary linear mapping from the latest row's numerical features to the fractional score. It can capture additive trends but does not enforce a forgetting curve. Predictions are clipped to the valid 0-to-1 range during serving.

Random Forest

An ensemble of decision trees captures nonlinear feature interactions through recursive partitions. It uses only the current row's numeric feature vector, unlike the GRU which receives recent ordered event rows. It is a strong nonlinear tabular baseline.

GRU candidate

The GRU receives a sequence of up to 10 rows, each with five numerical features. It uses two recurrent GRU layers, hidden size 64, batch-first inputs and dropout 0.2, followed by Linear(64,32), ReLU and Linear(32,1) with sigmoid. Gating lets the recurrent state retain or replace information as each historical row is processed. The final hidden state maps to the predicted next-session score.

```
z_t = sigmoid(W_z x_t + U_z h_(t-1) + b_z)
r_t = sigmoid(W_r x_t + U_r h_(t-1) + b_r)
n_t = tanh(W_n x_t + b_n + r_t * (U_n h_(t-1) + b_hn))
h_t = (1 - z_t) * n_t + z_t * h_(t-1)
The reset gate r controls candidate-state history; update gate z controls how much prior state is
retained.
```

---

4. Training, split design and deployment selection

The training job streams the compressed source file in chunks. It retains a deterministic 1-in-100 learner hash sample, validates numeric counts and timestamps, forms next-event pairs, drops invalid targets and restricts candidate intervals to 0 < g <= 30. The reported comparison has 617 learners and 62,931 eligible pairs.

Split	Partitioning	Deployment question approximated
Forward-time	Target-event chronology, 70/15/15	How well will the model score later observed events for this sampled population when their earlier history exists?
Learner-held-out	Disjoint learner IDs, 70/15/15	How well on people absent from training, assuming each supplies their own history at inference?

Train fits model parameters. Validation supports model selection and early stopping. Test metrics are held out. Three seeds (42, 52, 62) are recorded. Learner-cluster bootstrap intervals resample learners, rather than treating each interaction as an independent person, to account for repeated observations within learner.

The artifact-selection rule chooses the candidate with the lowest average validation MAE across the two protocols. It selects HLR. The serialized next_session_champion.joblib contains the selected model kind and fitted parameters; next_session_service.py loads it once and dispatches to the corresponding inference implementation.

Champion is a selection rule result, not a universal winner. GRU test ECE/RMSE are better in some comparisons while HLR has lower MAE. The chosen objective does not directly score review-date accuracy.

---

5. Evaluation and actual recorded results

Model	Mean validation MAE across protocols	Forward test MAE	Forward test RMSE	Forward test ECE	Learner-held-out test MAE	Learner-held-out test RMSE	Learner-held-out test ECE
Mean	0.1791	0.1763	0.2798	0.0074	See JSON report	See JSON report	See JSON report
HLR	0.1304	0.1404	0.2926	0.0782	0.1001	0.2591	0.0695
Linear	0.1725	0.1748	0.2763	0.0130	See JSON report	See JSON report	See JSON report
Random Forest	0.1722	See JSON report	See JSON report	See JSON report	See JSON report	See JSON report	See JSON report
GRU	0.1719	0.1711	0.2758	0.0091	0.1530	0.2474	0.0105

MAE is the average absolute score error. RMSE is the square root of average squared error and penalizes larger misses more. ECE groups predictions into ten equal-width score bins and averages the difference between predicted and observed score per bin, weighted by bin population. Lower is better for all three, but MAE, RMSE and ECE answer different questions.

The validation MAE table is the champion selection basis. It is not valid to infer HLR is better on all outcomes: in the displayed forward-time results the GRU has lower RMSE and ECE. The JSON report contains per-seed values and learner-cluster confidence intervals. The UI labels forecasts experimental and uncalibrated.

---

6. Serving path: from event to review date

LearningEvent rows ordered by time	App adapter adds prior gaps	Candidate gaps 0.25 to 30 days	HLR score per gap	Policy 80% threshold	RevisionSchedule calendar date

The app API's next_session.py creates one ordered event list for a topic. The last event's gap is not observed yet; each hypothetical future gap is supplied to /predict/next-session/scenarios. The ML API runs the champion for each candidate gap and returns score, gap, model kind, target type, and uncalibrated/experimental flags.

The date policy selects the latest candidate delay whose predicted score is at least 0.80; if none meets it, the earliest candidate is selected. A separate dashboard interpolation estimates where the sampled model curve crosses 0.80. These are related but distinct policy views. Auto-scheduling writes a pending RevisionSchedule only if the model forecast is available and the topic does not have a pending cycle.

The reminder is an estimated threshold-crossing signal. It is not the time at which forgetting was observed, and the 80% threshold is an app policy choice. The public trace task needs app-domain delayed assessments before this date can be validated for course concepts.

---

7. Limits and the right next experiment

- The source task is a language-learning next-session score, not a delayed retention study. Lexeme IDs do not map to app course-topic IDs. Focus minutes are not training features or labels.
- Current metrics do not demonstrate the model's crossing-date error on app topics. Evaluate a proper time-to-review target before claiming reliable forgetting dates.

- The forecast is not calibrated for long-term memory. ECE on proxy next-session outcomes does not establish calibration on user course topics.

A useful pilot needs consented learners, course-aligned questions and delayed assessments; fixed schedule versus adaptive policy; participant-level outcomes; held-out future assessment content; preregistration and a power calculation for a confirmatory claim. Existing user accounts are not automatically research consent.

Repository traceability and source files

The report describes the local repository snapshot, not an independently deployed cloud instance. A running deployment can differ if its backend, database migrations, model artifacts or environment variables have not been updated.

Primary source files reviewed:

- docs/MODEL_ARCHITECTURE.md
- docs/DATASET_AND_PREPROCESSING.md
- docs/EXPERIMENT_PROTOCOL.md
- docs/RESULTS.md
- docs/RESEARCH_LIMITATIONS.md
- ml/src/next_session_model.py
- ml/src/next_session_service.py
- ml/src/train_next_session_model.py
- app/services/next_session.py
- app/routers/recommendations.py